NAMP: A Network-Aware Model Partitioning Framework for Constrained Devices

Fernando Rego
fernando.rego@fraunhofer.pt
Fraunhofer Portugal AICOS
Porto, Portugal

João Oliveira
Filipe Sousa
joao.oliveira@fraunhofer.pt
filipe.sousa@fraunhofer.pt
Fraunhofer Portugal AICOS
Porto, Portugal

Luis Almeida
lda@fe.up.pt
CISTER / FEUP - University of Porto
Porto, Portugal

Abstract

Despite the growing computing capacity of sensors, enabling them to execute Machine Learning (ML) models without requiring cloud resources is still a challenge given their limited computational power, memory, and energy. Current ML frameworks for developing, deploying and executing ML models typically overlook these limitations and consider monolithic models that run in single-devices. This paper proposes the Network-Aware Model Partitioning (NAMP) framework that allows breaking neural networks into smaller submodels considering the resources available in the system. The submodels are deployed as services on a network of constrained devices to provide distributed inferences. Two emulated scenarios with 15 devices with limited memory show NAMP deploying a distributed LeNet-based model that would be unfeasible if monolithic. Comparative analysis shows the NAMP-derived solution minimises communication overhead, increasing inference efficiency.

Keywords

TinyML, Internet of Things, Distributed Inference, Model Partitioning, Neural Networks

ACM Reference Format:
Fernando Rego, João Oliveira, Filipe Sousa, and Luis Almeida. 2025. NAMP: A Network-Aware Model Partitioning Framework for Constrained Devices. In *The 4th International Workshop on Real-time and IntelliGent Edge computing (RAGE '25), May 6–9, 2025, Irvine, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3722567.3727844

RAGE '25, Irvine, CA, USA

ACM ISBN 979-8-4007-1611-9/2025/05
https://doi.org/10.1145/3722567.3727844

1 Introduction

Smart systems are becoming increasingly common in our daily lives, from sensors and health-monitoring devices to smart homes. These systems leverage distributed computing environments comprised of sensors and smart devices, often heterogeneous and with severe resource limitations, to monitor the environment in which they operate.

The emergence of TinyML facilitated Machine Learning (ML) in such devices, enabling local inference and reducing dependency on cloud services. This brings advantages such as data locality, faster response, low energy consumption and cost reduction [9]. Though lightweight ML models are provided by current TinyML frameworks, memory and computational power constraints narrow the supported models.

This paper proposes the Network-Aware Model Partitioning (NAMP) framework that partitions neural networks (NN) into smaller deployable submodels that are distributed across devices. NAMP leverages a network of resource-constrained devices to run NN models that would not fit in any device alone in a monolithic form. We overview the framework and present evaluation results using emulated scenarios, highlighting aspects for further development and refinement.

In the rest of the paper we review related work on TinyML frameworks and model partitioning in Section 2. Section 3 introduces NAMP, while Section 4 covers our experimental setup, evaluation scenarios, and discusses the results. Section 5 concludes the paper and presents future work.

2 Related Work

This brief survey focuses on splitting NN models and on ML frameworks that consider the resource constraints imposed by the devices to leverage or support partitioning methods.

2.1 Model Partitioning Methods

Partitioning monolithic NNs for deployment in a network of constrained devices can follow two main approaches:

2.1.1 Per-layer partitioning. According to this approach, the NN is seen as a set of layers that can be grouped in different subsets, representing the submodels that can be deployed on distinct devices. TinyML frameworks usually have good support and flexibility at the layer level, facilitating the manipulation and extraction of individual or groups of layers. However, this partitioning approach is always tied to the size of the layers. Therefore, even though memory is effectively reduced when using per-layer partitioning, layers with significant sizes can still lead to challenges regarding the memory constraints imposed by the devices. To mitigate this challenge, Stahl et al. [10] propose partitioning techniques that reduce the memory requirements of each submodel when performing a distributed inference. The Layer Output Partitioning (LOP) technique allows the replacement of a fully connected (FC) layer by sub-layers that can be parallelised and distributed across devices. The final layer output results from concatenating all outputs from the sub-layers.

2.1.2 Graph-based partitioning. In this case, the NN is perceived as a graph, with each vertex representing either a single neuron, a group of neurons, or an entire operation, while the edges correspond to the data transfers between the vertices. By representing an NN as a graph, the partitioning problem can be transformed into a graph partitioning problem. This approach can operate at the neuron level within the NN and is not dependent on the entire layer. Therefore, depending on the definition of the vertices, graph-based partitioning can be more flexible than per-layer partitioning, resulting in diverse solutions. Nevertheless, there are difficulties associated with this approach as the TinyML frameworks usually represent NNs as a task graph, where the smaller task graph unit can represent an ML operation or even an entire layer, therefore lacking support at the neuron level.

2.2 Frameworks and Partitioning

Current ML frameworks such as TensorFlow [1] and PyTorch [8] facilitate model development but are often unsuitable for IoT due to their focus on monolithic models and limited support for constrained devices. Consequently, we focused on proposals that support constrained devices.

TensorFlow Lite for Microcontrollers (TFLM) [3] enables ML on embedded systems by converting models into a compact format that reduces memory usage and improves inference efficiency. However, partitioning layers in TFLM has to be done manually. DIANNE [2] allows developing and distributing neural networks across heterogeneous devices but requires users to assign modules to specific devices, lacking automated partitioning tools. AR-MDI [6] proposes a decentralised approach for distributing model layers based on computational and transmission delays. Nevertheless, it requires the execution of a model allocation algorithm, which is challenging when using resource-constrained devices.

Graph-based libraries like METIS [4] and works such as RaNNC [11] support NN partitioning but generally target larger models and devices with significant resources.

Among all the frameworks we surveyed, none optimises ML model distribution considering the resources available in networks of resource-constrained devices. To address this gap we propose NAMP, a framework that we describe and assess in the remainder of the paper.

3 Network-Aware Model Partitioning

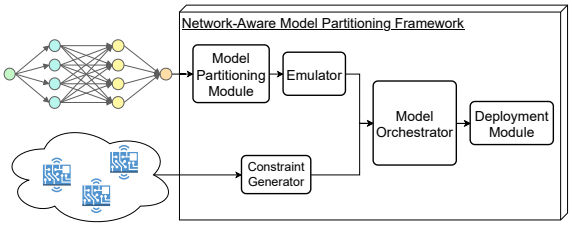

Figure 1: High-level overview of Network-Aware Model Partitioning Framework.

Creating network-aware ML models is crucial to use network resources efficiently. Therefore, we propose the NAMP framework, which provides model partitioning and deployment mechanisms to deploy and run ML models on a network of resource-constrained devices considering dynamic information about the available network resources. Figure 1 shows a high-level overview of the NAMP framework.

3.1 Model Partitioning Module

The Model Partitioning Module implements a per-layer partitioning approach that creates submodels by splitting the NN into separate layers or groups of layers. To mitigate the layer size limitation we combine a partitioning method similar to the LOP technique [10] that can replace a layer with multiple smaller layers containing only a section of the entire fully connected (FC) layer parameters. However, this technique partitions FC layers, only. Non-FC layers can only be segregated in a submodel using the per-layer partitioning approach with the associated submodel size limitation.

NAMP accepts an NN in TensorFlow or ONNX[1] format as input *model* and uses the algorithm in Listing 1 to partition the model leveraging framework tools to extract all possible *submodels* while finding replacements for all FC layers.

The algorithm iterates through the L layers in the model to extract all possible submodels. For each group of layers between layers i and j an object is created to store the extracted

[1]Linux Foundation, ONNX, https://onnx.ai/

Listing 1: Model partitioning algorithm.

```
function ModelPartitioning(model)
  L ← model.layers
  submodels ← []
  for i ← 0 to |L| do
    for j ← i to |L| do
      submodelName ← str(i) + "-" + str(j)
      submodel ← ExtractSubmodel(model,i,j)
      tfliteModel ← ConvertToTflite(submodel)
      SaveTFLiteModel(tfliteModel, submodelName)
      if i = j and L[i] is FCLayer then
        submodel.FCReplacements ← GetFCReplacements(submodel)
      end if
      submodels.add(submodel)
    end for
  end for
  return submodels
end function
```

Listing 2: NAMP framework API.

```
class Constraint:
  abstract void init(args)
  abstract bool validate(model)

class Device:
  list[Constraint] constraints
  abstract void init(args)
  abstract list[Constraint] collect_constraints()
  abstract bool validate_constraints(model)

class DeviceNetwork:
  list[Device] devices
  abstract void init(args)
  abstract list[Device] collect_devices()
  abstract bool deploy(model_assignments)

class Heuristic:
  abstract void set_weights(graph, devices)
```

submodel and possible replacements if it corresponds to an FC layer. Finally, since we assume that the constrained devices use the TFLM interpreter, all submodels are converted to TFLM format and stored in a *.tflite* file.

3.2 Emulator

Although the algorithm presented in the previous section performs model partitioning, the only information stored about each submodel is the index of the first and last layer processed by the submodel relative to the original model. However, NAMP needs additional information, such as the resources required to execute each submodel, which depends on the model inference runtime. To collect this information, we use an emulator that executes and analyses each submodel and any existing replacements.

Since we are focused on constrained devices, we implemented a firmware application built on top of the Zephyr RTOS[2] containing the TFLM interpreter and we emulate it using QEMU. This application gathers the information of each submodel, such as model size and required memory for the TFLM tensor arena. This approach allows us to get accurate resource requirements since the code we are emulating is the code that will eventually be deployed on real devices.

3.3 Constraint Generator

Network awareness is a crucial aspect of the NAMP framework to efficiently distribute the generated submodels across devices according to their resources. This module acquires information from the devices in the network to generate the constraints that the Model Orchestrator must respect.

The network resources information is acquired by NAMP when the framework is executed. After deploying a model, changes in network resources must trigger a new orchestration cycle to accommodate the updated information.

[2]Linux Foundation, Zephyr Project, https://www.zephyrproject.org/

We define an API, shown in Listing 2, that implementations must follow to provide the required data and ensure compatibility with the NAMP framework. An implementation must include the *DeviceNetwork* class that provides the list of *Devices* to the framework, abstracting the device discovering procedure. Each *Device* contains a list of *Contraints* that will evaluate an eventual model assignment in a pass/fail way by the Model Orchestrator. Furthermore, by defining the *DeviceNetwork*, *Device* and *Constraints* classes abstractly, we can ensure that the framework is independent of the network intricacies, such as the communication technology or protocol and easily extend NAMP to integrate additional information and new constraints.

3.4 Model Orchestrator

We model the orchestration of the models across the available devices as a pathfinding problem with constraints in which the generated path, i.e., the graph of submodels to be deployed in the devices, must sequentially process all layers from the original model and satisfy all generated constraints. The graph created to solve this pathfinding problem contains nodes representing the submodels and edges defining the connections between the sequential submodels.

Since the final solution must be optimised considering the obtained network information, the framework API, shown in Listing 2, defines the class *Heuristic* with the method *set_weights*, which receives the graph containing all submodels and the network devices. This method is responsible for assigning a weight to each edge of the graph following any heuristic based on the requirements of each submodel and the available resources from each device.

Currently, our optimisation target is to minimise the data transfers required in a distributed inference. Therefore, each edge is weighted based on the number of bytes a device must transmit to the device hosting the next submodel.

Listing 3: Backtracking algorithm for submodel sequence validation and assignment.

```
1  function ValidateAndAssignSequence(S,devices,index,A)
2    if index = |S| then
3      return True
4    end if
5    for each device ∈ devices do
6      if device.ValidateConstraints(S[index]) then
7        A[index] ← device
8        device ← UpdateDevice(device,S[index])
9        if ValidateAndAssignSequence(S,devices,index+1,A) then
10         return True
11       end if
12       device ← RestoreDevice(device,S[index])
13       A.Delete(index)
14     end if
15   end for
16   for each R ∈ S[index].FCReplacements do
17     B ← {}
18     if ValidateAndAssignSequence(R,devices,0,B) then
19       A[index] ← B.Values()
20       if ValidateAndAssignSequence(S,devices,index+1,A) then
21         return True
22       end if
23       devices ← RestoreDevices(devices,R,B)
24       A.Delete(index)
25     end if
26   end for
27   return False
28 end function
```

Finally, the pathfinding problem with constraints can be solved by finding the shortest path in the weighted graph that satisfies every network constraint. Consequently, the Model Orchestrator employs Yen's algorithm [12] that obtains the k shortest paths and a backtracking algorithm, presented in Listing 3, that tries to assign all submodels contained in the k shortest path to a device while verifying that the solution satisfies all imposed constraints. This approach executes Yen's algorithm until the backtracking algorithm identifies the first shortest path that satisfies every constraint. The Model Orchestrator returns the set of submodel-to-device assignments that can be deployed, enabling the distributed inference.

3.5 Deployment Module

The framework last component is the deployment module that deploys the final graph of submodels across the network of devices according to the Model Orchestrator output. Since the deployment procedure depends on the target devices, we defined the *deploy* method to the *DeviceNetwork* class in the NAMP framework API. This method receives the output of the Model Orchestrator containing the model-to-device assignments and the device-to-device linkage.

4 Evaluation

We evaluate our proposal by emulating a constrained network scenario and requesting the NAMP framework to deploy an ML model. We analyse the solution provided by the framework and compare it with alternative execution paths to understand its performance.

Listing 4: Device CoAP API.

```
1  /info # Request device information [GET]
2  /inference # List available inferences or create an inference
     instance [GET|POST]
3    /[id] # Request/Delete inference instance [GET|DELETE]
4      /model # Upload model [POST]
5      /input # Retrieve/Update model input [GET|POST]
6      /output # Retrieve model output in binary [GET]
7      /observe # Add/Remove resource observer [POST|DELETE]
```

4.1 Experimental Setup

As explained in Section 3, NAMP framework must be capable of communicating with the devices in the network and requires an implementation following the NAMP API.

At the device level, we implemented a firmware application using the Zephyr RTOS that includes a simple CoAP communication protocol and the TFLM interpreter. The implemented CoAP API is shown in Listing 4 and provides methods to collect device information, create inferences, post input data, and get output data. Additionally, we enable observer support to allow us to use the output of a device as the input of another device, thus enabling the automatic inference of each submodule.

At the framework level, the available devices are provided using a JSON file that details the IP addresses of all devices. Therefore, the *collect_devices* method (cf. Listing 2, line 16) reads the file and creates the *Devices*. Using the IP address of each device, we generate the list of constraints by sending a GET request to the CoAP API endpoint */info*, which currently returns the available heap memory, the number of free inference slots, and the configured arena size for the tensors in TFLM. This information is transformed into a set of *Constraints* for each device that validates if a model can be deployed in the device. The *deploy* method (cf. Listing 2, line 17) leverages the CoAP endpoints in */inference* to deploy each submodule and configure the correct sequence of observers.

For the tests, we used the IoTNetEmu tool [7] to emulate the network of devices. This tool creates a set of emulated devices running applications according to the user configuration. We leveraged Zephyr and IoTNetEmu capabilities to emulate Cortex-M3 nodes using QEMU and connect them via Linux network bridges. Finally, we used a non-quantized model based on LeNet [5] as the input for NAMP. This model is a convolutional NN trained to recognise handwritten characters and is publicly available on TensorFlow Hub[3]. Table 1 shows a summary of the layers that compose the model. It is worth noting that layer size refers to the size of the layer when converted to a submodel.

[3]https://www.kaggle.com/models/fernandorego0/lenet

Table 1: LeNet Layers Overview.

Layer	Layer Type	Layer Size (B)	Tensor Arena Size (B)	Inputs	Outputs
0	Conv	1776	17536	784	3456
1	Pooling	912	17760	3456	864
2	Conv	10808	8192	864	1024
3	Pooling	912	5600	1024	256
4	Flatten	968	1496	256	256
5	FC	124436	1984	256	120
6	Flatten	952	1408	120	120
7	FC	41732	1296	120	84
8	Flatten	952	1120	84	84
9	FC	4672	976	84	10

4.2 Results

To evaluate the NAMP framework, we configured a scenario comprising 15 homogeneous devices with 64 KiB of allocatable memory and a 20 KiB tensor arena (Scenario 1) emulated using IoTNetEmu. Figure 2 shows the solution provided by NAMP when requested to deploy the LeNet-based model in the emulated network. Each circle represents a different device and its assigned submodel, identified by a name in the format $x - y$, where the submodel $x - y$ takes as input the expected values by the layer with index x and executes the set of layers until the layer with index y, inclusively.

The NAMP solution required five devices, where the first device processes the first five layers (0-4), three devices process layer 5 and the final device merges all outputs from the previous devices and executes the remaining layers (6-9). This is the expected solution since our optimisation target is to minimise the data transfers in the distributed inference while satisfying the imposed network constraints. The main limitation of this scenario is the required memory for layer 5, which includes its size (~122 KiB), the memory for the communication of the inputs and outputs (~3 KiB per device), and the overhead associated with splitting and converting the layer to smaller TFLM submodels (~2 KiB), requiring three devices to process the entire layer, given the 64 KiB of allocatable memory in each device.

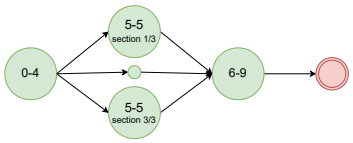

Figure 2: NAMP solution for Scenario 1. Only devices used in the solution are represented.

To evaluate the performance and efficiency of the employed heuristic, we compared the NAMP solution with the next three shortest paths and with three different valid paths chosen randomly by measuring the amount of transferred data and the distributed inference time. The inference was executed 1000 times for each solution, and outliers were identified and removed due to the device emulation, where discrepancies and time variations may appear as they depend on the system scheduler. Table 2 shows the collected results.

Table 2: Comparison of the solution generated by NAMP for Scenario 1 to other valid solutions.

Solution	Devices	Data Transferred (B)	Average Inference Time (ms)
NAMP	5	3592	89,23
ShortestPath #2	6	3928	96,03
ShortestPath #3	6	3928	95,56
ShortestPath #4	6	4072	95,99
RandomPath #1	9	8888	136,51
RandomPath #2	11	23048	248,06
RandomPath #3	7	4952	105,08

Results show that the data transfers impact the execution time of each distributed inference as expected. Therefore, if the implemented heuristic does not consider the required communications, the path can lead to excessive data transfers, such as RandomPath #2, where 23048 bytes are transmitted to complete a single model inference, resulting in an execution time almost three times larger than the NAMP solution. Among these solutions, the NAMP solution transfers the least amount of data and is the fastest to complete the distributed inference. The next best solution already requires more devices and transfers at least 336 more bytes to complete the inference. An efficient model distribution, which is highly affected by the implemented heuristic, is crucial to minimise communications and, consequently, reduce the execution time of each inference in the tested scenarios.

Nevertheless, it is vital to note that the network of devices was emulated in the test machine, leading to minimal communication overhead and marginal submodel inference time. Therefore, the transition to a real-world scenario brings several consequences, such as the communication overhead due to the usage of low-bandwidth network protocols and low-computing power devices. These challenges increase the importance of an adaptable framework, such as NAMP, where different optimization targets and heuristics that directly impact the model orchestration can be implemented.

To further evaluate NAMP partitioning capability, we created a heterogeneous environment by increasing each emulated device's constraints (Scenario 2). One device (D1) was configured with 12 KiB for the heap memory and 20 KiB for the tensor arena, and a second device (D2) with 20 KiB for the heap memory and 12 KiB for the tensor arena. The remaining devices (D3-D15) were configured with 20 KiB for the heap memory and only 4 KiB for the tensor arena.

Despite the constrained environment, NAMP successfully provided a solution, shown in Figure 3, to deploy the LeNet-based model across the emulated network. The solution now

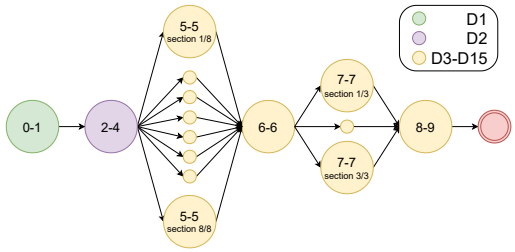

Figure 3: NAMP solution for Scenario 2.

requires all 15 devices and allocates the first two layers (0-1) to D1, the only device capable of running these layers due to tensor arena requirements. Layers 2 to 4 were deployed to D2, which is the only node that can fit layers 2 and 3 since D1 is already full. The partitioning of layer 5 requires 8 of the remaining devices since 6 devices are insufficient and the workload (120 neurons) is not equally divisible by 7 devices. Layer 7 is the second biggest layer, requiring 3 devices. With this scenario, we show how NAMP can be used to deploy a model even on a network of very constrained devices.

5 Conclusion

This paper introduced NAMP, a framework currently under development for partitioning neural networks into smaller, deployable submodels suited for networks of constrained devices. NAMP utilises device-specific information to optimise resource usage and reduce distributed inference time. The proposed algorithm models the partitioning as a pathfinding problem with constraints, aiming to minimise communications during a distributed inference.

Experimental results indicate that, in two simulated scenarios with 15 devices with limited memory, NAMP enables the deployment of a LeNet-based model, which a monolithic approach would have been unable to achieve. Additionally, comparative analysis shows that the NAMP-derived solution minimises communication overhead, leading to improved inference efficiency.

Future work will aim at expanding the available orchestration algorithms and optimization targets to enhance NAMP applicability, integrate IoT protocols such as LwM2M and MQTT for compatibility with existing IoT ecosystems, and evaluate NAMP in practical scenarios with defined use cases. Additionally, we plan to explore partitioning methods for different types of layers to overcome the current NAMP limitation of only dividing FC layers, and assess the deployment of different types of NN models, including LSTMs. Finally, integrating the NAMP framework in an IoT orchestration system could enable dynamic model redeployment in response to network resource changes, bringing increased value to IoT environments.

Acknowledgments

This work has received funding from the Horizon Europe research and innovation programme of the European Union, under grant 101092912, project MLSysOps.

References

[1] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, Derek G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USA, 265–283.

[2] Elias D. Coninck, T. Verbelen, B. Vankeirsbilck, S. Bohez, S. Leroux, and P. Simoens. 2015. DIANNE: Distributed Artificial Neural Networks for the Internet of Things. In *Proceedings of the 2nd Workshop on Middleware for Context-Aware Applications in the IoT* (Vancouver, BC, Canada) *(M4IoT 2015)*. Association for Computing Machinery, New York, NY, USA, 19–24. doi:10.1145/2836127.2836130

[3] R. David, J. Duke, A. Jain, Vijay J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes. 2021. TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems. In *Proceedings of Machine Learning and Systems*, Vol. 3. 800–811. doi:10.48550/arXiv.2010.08678

[4] G. Karypis and V. Kumar. 1998. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. *SIAM Journal on Scientific Computing* 20, 1 (1998), 359–392. doi:10.1137/S1064827595287997

[5] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. doi:10.1109/5.726791

[6] P. Li, E. Koyuncu, and H. Seferoglu. 2023. Adaptive and Resilient Model-Distributed Inference in Edge Computing Systems. *IEEE Open Journal of the Communications Society* 4 (2023), 1263–1273. doi:10.1109/OJCOMS.2023.3280174

[7] J. Oliveira, F. Sousa, and L. Almeida. 2023. IoTNetEMU - A Framework to Emulate and Test IoT Applications. In *Proceedings of the 19th ACM International Symposium on QoS and Security for Wireless and Mobile Networks (Q2SWinet '23)*. Association for Computing Machinery, New York, NY, USA, 33–37. doi:10.1145/3616391.3622774

[8] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library.* Red Hook, NY, USA.

[9] Ramon Sanchez-Iborra and Antonio F. Skarmeta. 2020. TinyML-Enabled Frugal Smart Objects: Challenges and Opportunities. *IEEE Circuits and Systems Magazine* 20 (2020), 4–18. Issue 3. doi:10.1109/MCAS.2020.3005467

[10] R. Stahl, Z. Zhao, D. Mueller-Gritschneder, A. Gerstlauer, and U. Schlichtmann. 2019. Fully Distributed Deep Learning Inference on Resource-Constrained Edge Devices. In *Embedded Computer Systems: Architectures, Modeling, and Simulation*. Berlin, Heidelberg, 77–90. doi:10.1007/978-3-030-27562-4_6

[11] M. Tanaka, K. Taura, T. Hanawa, and K Torisawa. 2021. Automatic Graph Partitioning for Very Large-scale Deep Learning. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 1004–1013. doi:10.1109/IPDPS49936.2021.00109

[12] Jin Y. Yen. 1970. An algorithm for finding shortest routes from all source nodes to a given destination in general networks. *Quart. Appl. Math.* 27, 4 (1970), 526–530. doi:10.1090/qam/253822